AI-Powered Event & Activity Discovery Platform

Full Project Specification • MVP • Architecture • Database • Team • Data Sources • APIs • 8-Week Plan

Project goal	Build a trustworthy, AI-powered discovery engine for events and activities that can later expand from one city to multiple locations.
Core idea	Users describe what they want in natural language; the system finds real events from structured data, ranks them, and explains the recommendations.
Primary users	People looking for concerts, exhibitions, workshops, nightlife, sports, festivals, talks, markets, and other local activities.
MVP principle	Build the data pipeline first and the AI second. AI must operate on real database records and must never invent event information.

Working principle: Keep the first release narrow enough to finish, but architect the system so adding another city or country later does not require rebuilding the product.

1. Product Overview

One-line pitch: An AI-powered discovery platform that helps people find what is happening around them based on interests, budget, time, location, and mood.

Problem: Event information is fragmented across websites and social platforms. Users often have to search multiple sources, compare dates and prices, and decide which activities actually match their preferences.

Solution: Aggregate public event information into one normalized dataset, then combine structured filtering, semantic search, ranking, and LLM-powered explanations.

Expansion: The product should not be hard-coded around Tbilisi. Location is a data attribute, not the product identity. The same system should support additional cities through venues, coordinates, sources, and city-level configuration.

MVP success criteria

- A user can enter a natural-language request such as: “I want something social this Friday evening under 30 GEL.”
- The system converts the request into structured filters.
- The backend searches real events stored in PostgreSQL/PostGIS.
- The system ranks the best candidates using semantic and structured signals.
- Every result contains verifiable event information and a source/official URL.
- Users can open an event, view its location on a map, and save it.
- The system can ingest and normalize events from at least two reliable sources.

2. Exact MVP Scope

Included in MVP

Feature	MVP requirement
Event discovery	Search and browse concerts, parties, exhibitions, theatre, workshops, tech events, sports, festivals, markets, family activities, talks, etc.
Natural-language search	User describes desired activity in ordinary language; LLM converts the request into structured filters.
AI recommendations	Return ranked real events with a short explanation of why each result fits.
Filters	Date, time range, category, maximum price, location, and optional keywords.
Map	Display event/venue locations using OpenStreetMap + Leaflet.
Event details	Title, description, date/time, venue, address, price, category, source, image where available, official link.
Saved events	Authenticated users can save and unsave events.
Data pipeline	Automated ingestion, parsing, normalization, validation, deduplication, expiry handling, and source provenance.
Vector search	Generate embeddings for event text and use pgvector for semantic candidate retrieval.
Basic analytics	Track event views, saves, shares/external clicks, and search activity.

Explicitly out of MVP

- Native iOS/Android application.
- Ticket purchasing or payment processing.

- Event marketplace for organizers.
- Full social network or friend system.
- Real-time user location tracking.
- Advanced multi-stop itinerary generation.
- Complex predictive popularity models.
- Large conversational chatbot with memory.
- Do not block launch on perfect personalization; basic interaction signals are enough for MVP.

3. User Flow & AI Architecture

Core flow: User query → LLM structured extraction → database/vector candidate retrieval → ranking → recommendation explanation → event details/source.

Example query

```
“I have 30 GEL and I’m free Friday night. I want something fun and social, preferably music or a party.”
```

LLM output

```
{ "date": "Friday", "time_range": "evening", "max_price": 30, "categories": ["music", "social"], "location": null, "keywords": ["fun"] }
```

Architecture layers

Layer	Responsibility
1. Query understanding	LLM extracts date, time, budget, categories, location, keywords, and other supported constraints into validated JSON.
2. Structured retrieval	PostgreSQL/PostGIS applies hard constraints such as date, price, category, city, and geographic distance.
3. Semantic retrieval	pgvector compares the query embedding with event embeddings to find conceptually relevant events.
4. Ranking	Combine semantic similarity with price, date, category, distance, and popularity signals.
5. Explanation	LLM generates a concise explanation using only the selected database record; it must not add unsupported facts.
6. Presentation	Frontend displays ranked event cards, filters, map location, source, and event details.

Recommended ranking formula

```
score = 0.30 × semantic similarity + 0.20 × price match + 0.20 × date match + 0.15 × category match + 0.10 × distance + 0.05 × popularity
```

The weights are initial defaults, not permanent truth. During testing, compare recommendations against manually judged examples and adjust the weights.

Critical AI guardrail

Never allow the LLM to invent events, prices, dates, venues, or links. Candidate events must come from the database. The explanation layer receives the selected records as its factual context and should only summarize those records.

4. Database Schema

Recommended database: PostgreSQL with PostGIS for geographic queries and pgvector for embeddings.

Table	Key fields	Purpose
events	id, title, description, short_description, category_id, venue_id, start_time, end_time, price_min, price_max, currency, image_url, source_url, source_name, language, age_restriction, status, created_at, updated_at	Canonical event record.
categories	id, name, description	Normalized event categories.
venues	id, name, address, latitude, longitude, neighborhood, website	Reusable venue/location data.
sources	id, name, url, source_type, last_scraped_at, status	Tracks ingestion sources and health.
event_sources	id, event_id, source_id, external_id, source_url, first_seen, last_seen	Links canonical events to source records and provenance.
users	id, email, name, created_at	Basic user account.
user_preferences	id, user_id, category_id, preference_score	Optional category preference signals.
saved_events	id, user_id, event_id, created_at	Saved/favorite events.
user_interactions	id, user_id, event_id, interaction_type, timestamp	View, click, save, share, external-click signals.
event_embeddings	event_id, embedding, model, created_at	Vector representation for semantic search.

Important relationships

- events.category_id → categories.id
- events.venue_id → venues.id
- event_sources.event_id → events.id
- event_sources.source_id → sources.id
- saved_events.user_id → users.id; saved_events.event_id → events.id
- user_preferences.user_id → users.id; user_preferences.category_id → categories.id
- event_embeddings.event_id → events.id

Data quality requirements: normalize time zones, prices, currencies, categories, venue names, and source URLs. Mark expired events rather than deleting them immediately so pipeline behavior can be audited.

5. Data Pipeline

Pipeline: Sources → collectors/scrapers/APIs → raw data → parsing → normalization → validation → deduplication → PostgreSQL/PostGIS → embeddings → search/recommendation layer.

Step	Output
Fetch	Raw source response/page/API payload.
Parse	Title, description, date/time, price, venue, category, URL, image, source metadata.
Normalize	Canonical date/time, currency, category, venue, price range, language, location.
Validate	Required fields, valid dates, plausible prices, valid URLs, future/active status.
Deduplicate	Merge records representing the same event across sources.
Geocode	Venue latitude/longitude for map and distance ranking.
Persist	Canonical event + provenance records in PostgreSQL.
Embed	Generate/update event embeddings when relevant content changes.

Step	Output
Expire	Automatically mark events as expired after their end time.

Initial deduplication: title similarity + venue + date/time. Later, embeddings can improve duplicate detection.

6. APIs & Data Sources

Internal REST API

Method	Endpoint	Purpose
GET	/events	Browse/filter events.
GET	/events/{id}	Get event details.
POST	/search	Natural-language search + ranked recommendations.
GET	/recommendations	Personalized/basic recommendations.
POST	/events/{id}/save	Save event.
DELETE	/events/{id}/save	Remove saved event.
GET	/saved-events	List user's saved events.
POST	/interactions	Record views, clicks, shares, etc.

Potential public data sources

Source	Potential use	Implementation note
Eventebi.ge	Concerts, theatre, raves, cultural events	Evaluate terms/robots and structure before ingestion.
Tbiliseli Events	Concerts, exhibitions, theatre, festivals, parties, workshops, sports	Useful multi-category source.
MyTbilisi	Exhibitions, festivals, film, food, music, sport, talks	Potential cultural/local discovery source.
Eventbrite	Public events where applicable	Prefer official APIs or permitted access.
Afisha	Exhibitions, lectures, performances and cultural events	Check access and reuse conditions.

Compliance: Verify each source's terms of use, robots rules, API availability, licensing, and attribution requirements. Do not bypass anti-bot systems. Store source URLs and provenance for every event.

7. Tech Stack

Layer	Technology	Why
Frontend	Next.js + TypeScript + Tailwind CSS	Fast modern web UI and reusable components.
Map	OpenStreetMap + Leaflet	Open map stack suitable for event locations.
Backend	Python + FastAPI	Simple, fast API layer aligned with the team's Python/data skills.
Database	PostgreSQL + PostGIS + pgvector	Relational data, geospatial queries, and vector search in one system.
Data pipeline	Python, httpx, BeautifulSoup, Pandas	Collection, parsing, cleaning, normalization.

Layer	Technology	Why
AI	LLM API + embeddings	Query extraction, explanations, semantic retrieval.
Auth	Simple JWT/session-based auth	Enough for saved events in MVP.
Deployment	Vercel + Render/Railway + managed PostgreSQL	Simple deployment path for a student team.
Version control	GitHub	Branches, pull requests, issues, project management.

8. Team Responsibilities

Member	Primary ownership	Deliverables
1. Data Engineer	Data ingestion + database	Collectors/APIs, raw parsing, normalization, deduplication, PostgreSQL schema, scheduled pipeline, data quality.
2. AI/ML Engineer	AI layer	LLM query parser, embeddings, semantic search, ranking, recommendation explanations, hallucination guardrails.
3. Backend Engineer	FastAPI + integration	REST endpoints, auth, search service, event service, database integration, interaction tracking.
4. Frontend Engineer	Web application	Homepage, search UI, event cards, filters, map, details page, saved events, responsive design.
5. Product / Full-Stack / QA	Integration + UX + quality	Requirements, user flow, testing, UI polish, integration, deployment support, documentation, demo/presentation.

Team rule: Every member should have one primary owner area, but integration is shared. No feature is considered complete until it works end-to-end on the deployed branch.

9. Eight-Week Development Plan

Week	Goal	Concrete deliverables
1	Foundation	Finalize requirements; wireframes; GitHub workflow; PostgreSQL schema; FastAPI skeleton; Next.js skeleton; local development setup.
2	Data ingestion	Connect first 2 sources; implement collectors; define canonical event format; load 100+ real events.
3	Data quality	Normalize dates/prices/categories; venue normalization; deduplication; expiry logic; source provenance; data validation tests.
4	Frontend MVP	Homepage; search/browse; event cards; event detail page; filters; map; responsive UI. Product should work without AI.
5	AI search	LLM structured query extraction; /search endpoint; hard filters; query validation; natural-language search UI.
6	Recommendations	Event embeddings; pgvector retrieval; ranking formula; explanation generation; AI evaluation examples.
7	Personalization	Saved events; interaction tracking; basic preferences; recommendation improvements; UI polish; error handling.
8	Launch + demo	Deployment; end-to-end testing; monitoring; README; architecture diagram; demo dataset; final presentation; bug fixing.

Weekly definition of done

- All work is committed through GitHub branches and reviewed before merge.
- Each week ends with a working integration demo, not only isolated components.
- Critical bugs and data-quality failures are tracked as GitHub issues.
- API contracts and schema changes are documented so frontend/backend/data work stays synchronized.

10. Testing & Evaluation

Area	Metric / test
Data quality	Missing-field rate, duplicate rate, invalid-date rate, ingestion success rate, source freshness.
Product	Searches, event views, saves, external clicks, recommendation click-through rate.
AI	Query parsing accuracy, recommendation relevance, search success rate, hallucination rate.
Reliability	API error rate, failed pipeline runs, stale-source detection.
Manual evaluation	Create 20–30 representative user queries and manually score whether returned events actually match the request.

For the final demo, prepare a small evaluation set containing easy, ambiguous, budget-constrained, date-constrained, and category-based queries. This makes the AI component measurable rather than purely visual.

11. Security, Reliability & Data Rules

- Validate all LLM output against a strict JSON schema before using it in database queries.
- Never execute raw LLM-generated SQL.
- Use parameterized database queries.
- Keep API keys and database credentials in environment variables/secrets, never in GitHub.
- Rate-limit public search endpoints if the deployment is exposed to the internet.
- Store source URLs and timestamps for traceability.
- Treat scraped content as untrusted input.
- Do not expose internal source credentials or private user data in logs.
- Provide clear attribution/links where source terms require it.

12. Future Roadmap

Version	Feature
V2	“Build My Evening” — generate a multi-stop plan based on time, budget, interests, and geographic proximity.
V2	Personal AI — “What would I like tonight?” based on saved events and interaction history.
V2	Group planning — combine preferences across friends.
V3	Popularity prediction and trend detection.
V3	Social discovery based on similar interests.
V3	Multi-city / multi-country expansion with localized sources and language support.
V3	Georgian-language natural-language search and multilingual event understanding.

13. Final Architecture Summary

Frontend: Next.js → FastAPI: REST/search services → PostgreSQL/PostGIS: structured event data + geographic filtering → pgvector: semantic retrieval → LLM: query parsing + grounded explanations.

Data side: Public sources → collectors → parsing/normalization → validation/deduplication → canonical events → embeddings → searchable database.

Core engineering principle: The AI is a layer on top of a trustworthy event dataset. The database is the source of truth; the LLM interprets requests and explains results but does not create event facts.

Recommended first milestone: By the end of Week 2, the team should be able to open a database containing at least 100 real, normalized events from two sources and retrieve them through the backend API. Once this works, the AI layer has a solid foundation.